

Informe técnico

VisionLite AI

I. INTRODUCCIÓN

Actualmente, muchas soluciones de visión artificial requieren modelos pesados, equipos con alta capacidad de procesamiento o procesos complejos de entrenamiento. Esto representa una dificultad para estudiantes, instituciones educativas o pequeños proyectos que desean implementar reconocimiento de objetos de manera rápida, económica y accesible.

Además, varias aplicaciones de inteligencia artificial visual necesitan analizar imágenes capturadas desde una cámara, pero hacerlo en tiempo real puede generar problemas de lentitud, alto consumo de memoria y baja eficiencia si se utilizan modelos demasiado grandes.

Por esta razón, surge la necesidad de desarrollar una aplicación ligera, intuitiva y funcional que permita reconocer objetos desde la cámara de un dispositivo, utilizando una técnica real de inteligencia artificial sin necesidad de entrenar un modelo desde cero.

II. OBJETIVOS

A. *Objetivo general*

Desarrollar una aplicación híbrida de visión por computador llamada **VisionLite AI**, capaz de reconocer objetos en vivo mediante la cámara de un dispositivo, utilizando el modelo CNN pre entrenado **MobileNet** y un backend desarrollado con **Flask y Python**.

B. *Objetivos específicos*

1) Implementar una interfaz gráfica con diseño móvil, usable e intuitivo.

2) Integrar una técnica real de inteligencia artificial basada en redes neuronales convolucionales.

- Utilizar MobileNet como modelo preentrenado para clasificación de imágenes.
- Capturar imágenes desde la cámara del dispositivo y enviarlas al backend para su análisis.
- Mostrar la predicción principal del modelo junto con su porcentaje de confianza.
- Visualizar las cinco predicciones principales mediante una gráfica de barras.
- Medir el tiempo de ejecución de cada inferencia.
- Realizar una experimentación básica comparando confianza, precisión y tiempo de respuesta.

III. METODOLOGÍA

El proyecto se desarrolló bajo el enfoque de Flask + app híbrida. La aplicación cuenta con una interfaz web adaptable a dispositivos móviles, desarrollada con HTML, CSS y JavaScript, y un backend en Python con Flask, encargado de procesar las imágenes y ejecutar el modelo de inteligencia artificial.

El funcionamiento general de la aplicación es el siguiente:

1. El usuario abre la aplicación desde el navegador del celular o computador.
2. La interfaz solicita permiso para acceder a la cámara.
3. La cámara captura fotogramas de forma automática cada cierto intervalo de tiempo.
4. Cada imagen capturada se envía al backend mediante una petición HTTP.

5. Flask recibe la imagen codificada en Base64.

6. La imagen se convierte a formato RGB y se redimensiona a 224x224 píxeles.

7. MobileNet procesa la imagen y genera una predicción.

8. El sistema devuelve la clase más probable, el porcentaje de confianza, las cinco predicciones principales y el tiempo de ejecución.

9. La interfaz muestra el resultado encima de la cámara y presenta una gráfica de barras con las probabilidades obtenidas.

La estructura general del proyecto fue la siguiente:

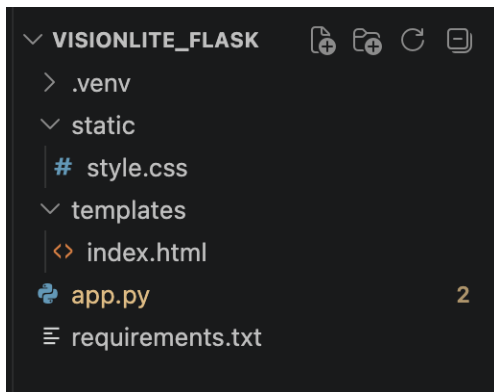


Ilustración I: Estructura del proyecto

El archivo **app.py** contiene la lógica principal de Flask y MobileNet.

El archivo **index.html** contiene la interfaz, el acceso a la cámara y las peticiones al backend.

El archivo **style.css** define el diseño visual de la aplicación.

El archivo **requirements.txt** contiene las librerías necesarias para ejecutar el proyecto.

Las tecnologías utilizadas fueron:

Tabla I: Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Python	Lenguaje principal del backend

Flask	Servidor web y conexión entre interfaz e IA
TensorFlow / Keras	Carga y ejecución de MobileNet
MobileNet	Modelo CNN preentrenado para clasificación
NumPy	Manejo de arreglos numéricos
Pillow	Procesamiento de imágenes
HTML	Estructura de la interfaz
CSS	Diseño visual móvil
JavaScript	Acceso a cámara y comunicación con Flask
Ngrok	Acceso HTTPS desde celular

4. Fundamentos teóricos

Redes neuronales convolucionales

Las redes neuronales convolucionales, conocidas como CNN, son modelos de aprendizaje profundo utilizados principalmente en la visión por computadora. Estas redes son capaces de analizar imágenes y extraer características visuales como bordes, formas, texturas y patrones complejos.

Una CNN trabaja mediante capas convolucionales que aplican filtros sobre la imagen para detectar características relevantes. A medida que la información avanza por la red, las primeras capas detectan patrones simples, mientras que las capas más profundas identifican objetos o estructuras más complejas.

MobileNet

MobileNet es una arquitectura de red neuronal convolucional desarrollada por Google en 2017. Su principal objetivo es ofrecer un modelo eficiente, rápido y ligero para dispositivos con recursos limitados, como smartphones, cámaras inteligentes y sistemas embebidos.

A diferencia de modelos más pesados como VGG16 o ResNet50, MobileNet utiliza convoluciones separables en profundidad, conocidas como depthwise separable convolutions. Esta técnica divide una convolución tradicional en dos operaciones:

1. Depthwise Convolution: aplica un filtro independiente a cada canal de entrada.
2. Pointwise Convolution: utiliza convoluciones 1x1 para combinar la información entre canales.

Gracias a esta estrategia, MobileNet reduce considerablemente el número de parámetros y operaciones computacionales, manteniendo una precisión aceptable.

La versión original, MobileNetV1, tiene aproximadamente 4.3 millones de parámetros y un tamaño cercano a 16 MB, lo que la hace adecuada para aplicaciones donde se requiere velocidad y bajo consumo de memoria.

Inferencia con modelo preentrenado

En este proyecto no se entrenó MobileNet desde cero. Se utilizó una versión preentrenada con el dataset ImageNet, el cual contiene millones de imágenes distribuidas en múltiples categorías.

El proceso realizado se conoce como inferencia, es decir, el modelo recibe una imagen nueva y genera una predicción con base en el conocimiento aprendido durante su entrenamiento previo.

Función Softmax

La salida final de MobileNet consiste en valores numéricos asociados a las diferentes clases. Para convertir estos valores en probabilidades se utiliza la función Softmax.

La fórmula de Softmax es:

$$P(\text{clase } i) = e^{z_i} / \sum e^{z_j}$$

Donde:

- z_i representa el valor de salida para una clase específica.
- e^{z_i} transforma ese valor en una escala positiva.
- $\sum e^{z_j}$ representa la suma de los valores exponenciales de todas las clases.
- $P(\text{clase } i)$ es la probabilidad asignada a una clase.

La clase con mayor probabilidad se toma como la predicción principal del modelo.

Accuracy

La accuracy o exactitud mide el porcentaje de predicciones correctas realizadas por el modelo.

Su fórmula es:

$$\text{Accuracy} = \text{Predicciones correctas} / \text{Total de predicciones}$$

Por ejemplo, si se prueban 10 objetos y el modelo acierta 8, la accuracy sería:

$$\text{Accuracy} = 8 / 10 = 0.8 = 80\%$$

En este proyecto, la accuracy puede calcularse manualmente realizando pruebas con diferentes objetos reales y comparando la predicción del modelo con el objeto observado.

5. Resultados

La aplicación VisionLite AI logró integrar correctamente una interfaz gráfica tipo móvil con un backend en Flask y el modelo MobileNet.

Los principales resultados obtenidos fueron:

- La aplicación permite acceder a la cámara desde el navegador.
- El sistema captura imágenes en vivo de manera automática.
- Flask recibe las imágenes y las procesa con Python.
- MobileNet genera una predicción sobre el objeto observado.
- El resultado principal se muestra encima de la cámara.
- Se visualiza el porcentaje de confianza de la predicción.
- Se muestran las cinco predicciones principales.
- Se agregó una gráfica de barras para representar las probabilidades.
- Se mide el tiempo de ejecución de cada inferencia.

Visualización de resultados

La aplicación muestra una gráfica de barras con las cinco clases más probables detectadas por MobileNet. Esta visualización permite interpretar mejor cómo el modelo distribuye su confianza entre varias posibles respuestas.

Por ejemplo, ante una imagen de una taza, el sistema podría mostrar resultados similares a:

Tabla II: Tabla de predicción de resultados

Predicción	Confianza
Coffee Mug	82%
Cup	9%
Water Bottle	4%
Bowl	3%
Plate	2%

Esta gráfica facilita el análisis de las métricas del modelo, ya que no solo muestra la predicción principal, sino también otras posibles clases consideradas por la red neuronal.

Tiempo de ejecución

El sistema también muestra el tiempo aproximado que tarda MobileNet en procesar cada imagen. Este valor permite evaluar el rendimiento de la aplicación y analizar si el modelo responde de forma adecuada para un entorno en vivo.

Ejemplo de registro experimental:

Tabla III: Tabla de registro experimental

Objeto real	Predicción	Confianza	Tiempo	¿Acertó?
Celular	Cellular Telephone	85%	950 ms	Sí
Taza	Coffee Mug	78%	900 ms	Sí
Botella	Water Bottle	72%	980 ms	Sí
Tecclado	Computer	88%	870 ms	Sí

Objeto real	Predicción	Confianza	Tiempo	¿Acertó?
	Keyboard			
Cuaderno	Book Jacket	55%	1020 ms	Parcial

Con base en pruebas como estas, se puede calcular una accuracy experimental. Por ejemplo, si en 5 pruebas el modelo acierta 4:

$$\text{Accuracy} = 4 / 5 = 0.8 = 80\%$$

Comparación de parámetros experimentales

También se puede analizar el comportamiento de la aplicación modificando parámetros como la calidad de la imagen o el intervalo de captura.

Tabla IV: Tabla de predicción de resultados

Configuración	Calidad JPEG	Intervalo de captura	Tiempo promedio	Observación
Prueba A	0.75	2 segundos	Mayor	Mejor calidad, más demora
Prueba B	0.45	1 segundo	Menor	Más rápida, menor peso de imagen

Esta comparación permite observar que una imagen de menor calidad reduce el tiempo de envío y procesamiento, aunque puede afectar la confianza de la predicción. Por otro lado, una imagen de mayor calidad puede mejorar el análisis visual, pero aumenta el tiempo de ejecución.

IV. CONCLUSIONES

El proyecto VisionLite AI permitió demostrar la aplicación práctica de una red neuronal convolucional preentrenada en un entorno funcional de visión por computador.

La aplicación cumple con el objetivo de reconocer objetos mediante la cámara de un dispositivo, utilizando MobileNet como modelo de inteligencia artificial. Además, la implementación con Flask permitió conectar la interfaz web con el backend en Python, haciendo posible procesar imágenes en tiempo real o en intervalos cortos.

MobileNet resultó ser una opción adecuada para este proyecto debido a su bajo tamaño, rapidez y eficiencia. Aunque no siempre ofrece la máxima precisión frente a modelos más grandes, su equilibrio entre rendimiento y consumo de recursos la convierte en una alternativa útil para aplicaciones móviles, educativas y sistemas con recursos limitados.

La incorporación de métricas como la confianza, el tiempo de ejecución y la gráfica de probabilidades permitió mejorar la interpretación de los resultados. Estas visualizaciones ayudan a comprender no sólo cuál fue la predicción principal, sino también qué otras clases fueron consideradas por el modelo.

En conclusión, VisionLite AI evidencia cómo una CNN preentrenada puede integrarse en una aplicación híbrida sencilla, visual e interactiva. El proyecto tiene potencial para ser aplicado en educación, agricultura, seguridad, robótica, smartphones e inspección visual básica.